Movie Recommendation System Using Collaborative Filtering

Taibaz Pathan
Faculty 2: Computer Science and Engineering
Frankfurt University of Applied Sciences
Frankfurt am Main, Germany
Student ID: 1564714
taibaz.pathan@stud.fra-uas.de
Supervisor: Prof. Dr. Andreas Pech

***Abstract*—This report presents the design, implementation, and evaluation of a movie recommendation system built on the MovieLens ml-latest-small dataset (100,836 ratings, 610 users, 9,742 movies). User-Based Collaborative Filtering (UBCF, Pearson correlation) and Item-Based Collaborative Filtering (IBCF, adjusted cosine similarity) were implemented without a recommender library and compared against three non-personalized baselines and a matrix factorization model (SVD). After hyperparameter tuning via grid search, UBCF achieved the lowest RMSE of the two neighborhood-based approaches (0.8430). SVD, run with default hyperparameters and not tuned, achieved the best overall performance on both accuracy and ranking metrics (RMSE = 0.8379, Precision@10 = 0.0601). A paired bootstrap test ($n = 1000$ resamples) indicated that UBCF's RMSE advantage over IBCF is larger than test-set sampling variation would explain (95% CI for the difference [0.0252, 0.0418], excluding zero). All personalized models outperformed the baselines on every metric, by a clear margin on RMSE. The study also examines cold-start behavior, sparsity sensitivity, and computational scalability.**

***Index Terms*—collaborative filtering, recommender systems, MovieLens, user-based CF, item-based CF, matrix factorization, SVD, RMSE, precision-recall**

I. Introduction

A. Motivation

Streaming catalogs now contain far more titles than a user can realistically browse, and many users abandon a session without picking anything simply because there are too many options. Recommender systems address this by using a user's past ratings, together with the ratings of similar users, to narrow a large catalog down to a short, relevant list. Collaborative filtering (CF) is one of the most common approaches to this problem and is used in production by companies such as Netflix and Amazon. Its main advantage over content-based methods is that it needs no manual tagging of items: relevance is inferred entirely from patterns in the user-item rating matrix.

B. Problem Statement

Given a sparse matrix of historical user ratings for movies, the task is to predict the rating a user would give to a movie they have not yet seen, and to use these predictions to build a ranked list of recommendations. Three things make this non-trivial: extreme data sparsity (98.3% of the user-item matrix is unobserved in the raw dataset), the cold-start problem for users and items with few ratings, and the fact that a low prediction error does not automatically mean a good ranked list, since accuracy and ranking quality have to be evaluated separately.

C. Objectives

- Implement the neighbor selection and prediction logic of User-Based and Item-Based Collaborative Filtering directly, rather than using a recommender library's built-in neighborhood models, and implement the similarity measures from scratch in NumPy.
- Evaluate both approaches using both rating-accuracy metrics (RMSE, MAE) and ranking-quality metrics (Precision@K, Recall@K, F1@K).
- Establish non-personalized baselines to quantify the actual value added by personalization.
- Tune model hyperparameters systematically via grid search rather than manual selection.
- Extend the comparison to a matrix factorization approach (SVD) as a third algorithmic family.
- Analyze system behavior under cold-start and sparse-data conditions, and characterize computational scalability.
- Produce a reproducible, tested, version-controlled implementation.

D. Report Structure

Section II reviews related work on collaborative filtering. Section III covers the dataset, preprocessing, the algorithms themselves, and how they were evaluated. Section IV presents the results: model comparison, a significance test, hyperparameter tuning, cold-start behavior, and scalability. Section V discusses two implementation issues found during development and the limitations of the current system, and Section VI concludes with directions for future work.

II. Background and Related Work

Collaborative filtering methods are broadly divided into memory-based and model-based approaches. Memory-based methods, which include both UBCF and IBCF, compute predictions directly from the raw rating matrix using similarity measures, without learning a compressed intermediate

representation. Resnick et al. [1] introduced the neighborhood-based formulation for user-based CF used in this project, in which a target user's rating is predicted as a similarity-weighted average of deviations from neighboring users' mean ratings. Sarwar et al. [2] later showed that item-based approaches, which compute similarity between items rather than users, offer comparable or superior accuracy while being more computationally stable, since item-item relationships change more slowly over time than user-user relationships. Herlocker et al. [3] examined the design choices within neighborhood-based methods and found that accuracy depends, among other factors, on the neighborhood size and on trusting similarities less when they rest on few co-rated items. The two hyperparameters tuned in this project, k and `min_support`, address these two choices, the latter as a hard cutoff rather than a weight. Model-based approaches, by contrast, learn latent factor representations of users and items; singular value decomposition (SVD), popularized during the Netflix Prize competition [4], is one of the most widely used techniques in this family and is included in this study as a point of comparison. For evaluation, Herlocker et al. [5] distinguish predictive accuracy metrics such as MAE and RMSE from classification accuracy metrics such as precision and recall, and Cremonesi et al. [6] showed that improvements in RMSE often do not translate into better top-N recommendations. Both kinds of metric are therefore reported in this study.

Although these methods are decades old, they remain relevant. Nikolakopoulos et al. [7] survey neighborhood-based methods and note that they are still widely used because of their simplicity, efficiency, and accuracy. Ferrari Dacrema et al. [8] examined 18 neural top-N recommendation algorithms from major conferences; of the 7 they could reproduce, 6 were often outperformed by comparably simple methods such as nearest-neighbor techniques. Rendle et al. [9] likewise showed that matrix factorization with properly selected hyperparameters outperformed a neural model proposed to replace it. Classical collaborative filtering is therefore still a meaningful subject of study, and these results show how strongly a comparison depends on how carefully each model is tuned.

III. Methodology

A. Dataset

The MovieLens ml-latest-small dataset [10] was used throughout this project. It contains 100,836 ratings on a 0.5–5.0 scale (in 0.5 increments), submitted by 610 users for 9,742 movies between March 1996 and September 2018. Exploratory analysis of the raw dataset revealed three properties with direct implications for model design. First, the mean rating (3.50) sits well above the scale midpoint (2.75), indicating a positive rating bias consistent with users self-selecting content they expect to enjoy; this motivated the use of mean-centered similarity measures rather than raw rating comparison. Second, both user activity and movie popularity are heavily right-skewed: the median user has rated only 68 movies while the most active user has rated over 2,000, and the median movie has only 3 ratings. Third, the raw user-item matrix is 98.3% sparse, meaning fewer than 2% of all possible (user, movie) pairs have an observed rating.

B. Data Preprocessing

To ensure that similarity computations were based on statistically meaningful sample sizes, an iterative filtering procedure was applied: users and movies with fewer than 20 ratings were removed, with the process repeated until convergence (since removing sparse movies can push previously-eligible users below the threshold, and vice versa). This reduced the dataset to 67,020 ratings across 566 users and 1,286 movies, lowering sparsity to 90.8% (92.6% for the training matrix the models are fitted on). The filtered data was then split per-user into training and test sets using an 80/20 ratio, yielding 53,614 training ratings and 13,406 test ratings. Splitting per-user, rather than globally at random, guarantees that every user in the test set has a corresponding profile in the training set, which is a prerequisite for meaningful CF evaluation. A fixed random seed (42) was used throughout for reproducibility.

C. Similarity Metrics

Cosine similarity and Pearson correlation were implemented from scratch in NumPy, without scikit-learn's pairwise-similarity utilities, both at the vector level (single pairs, missing ratings excluded) and at the matrix level (fully vectorized). Cosine similarity measures the angle between two rating vectors. Pearson correlation additionally mean-centers each vector, which corrects for users who rate systematically higher or lower than average. In the vector-level Pearson function the means are computed over co-rated items only, following Resnick et al. [1]; the matrix-level version instead centers each row by its overall mean, a common vectorized approximation. A minimum co-rating threshold (`min_support`) discards similarities computed from too few shared ratings, since these are statistically unreliable.

The two models use these components differently. IBCF uses the from-scratch matrix-level cosine function (Section III-E). The final UBCF model computes its Pearson matrix with pandas' `DataFrame.corr()`, which evaluates each pair over co-rated items only and returns no value for pairs with fewer than `min_support` co-ratings; the from-scratch Pearson functions were validated by unit tests but are not on UBCF's prediction path.

D. User-Based Collaborative Filtering (UBCF)

UBCF predicts a target user u's rating for item i as a similarity-weighted average of the deviations of u's k nearest neighbors from their own mean ratings, added back to u's own mean rating:

$$\hat{r}_{ui} = \bar{r}_u + \frac{\sum_v \mathrm{sim}(u,v) \cdot (r_{vi} - \bar{r}_v)}{\sum_v |\mathrm{sim}(u,v)|} \tag{1}$$

Candidate neighbors are restricted to users who have rated item i. The k candidates with the largest absolute similarity are selected, and only the positively correlated ones among them are kept, so the sum runs over at most k users. If fewer

than two valid neighbors remain, the model falls back to the user's mean rating $\bar{r}_u$. Because (1) is a weighted average of unbounded deviations rather than a convex combination of ratings, its output is not mathematically guaranteed to remain within the original [0.5, 5.0] rating scale (see Section V-A); predictions are therefore clipped to this range post-hoc.

E. Item-Based Collaborative Filtering (IBCF)

IBCF predicts a rating by finding the k items most similar to the target item that the target user has already rated, and combining the user's ratings for those items, weighted by item-item similarity. Following Sarwar et al. [2], item-item similarity was computed using adjusted cosine similarity: each rating is first centered by subtracting the rating user's mean (not the item's mean), which corrects for user-level rating-scale differences before comparing items. The prediction itself likewise uses item-mean-centered deviations, for the reasons discussed in Section V-A:

$$\hat{r}_{ui} = \bar{r}_i + \frac{\sum_j \mathrm{sim}(i,j) \cdot (r_{uj} - \bar{r}_j)}{\sum_j |\mathrm{sim}(i,j)|} \tag{2}$$

where j ranges over the k items rated by u with the largest absolute similarity to i. Unlike UBCF, negative similarities are kept. If none of these items has a non-zero similarity to i, the model falls back to the user's mean rating.

F. Matrix Factorization (SVD)

As a third, model-based point of comparison, a matrix factorization model was implemented using the SVD algorithm provided by the scikit-surprise library [11], configured with 50 latent factors and 20 training epochs (random seed 42). Despite its name, this algorithm, popularized by Simon Funk during the Netflix Prize, is not a truncated singular value decomposition of the rating matrix. It predicts

$$\hat{r}_{ui} = \mu + b_u + b_i + q_i^\top p_u, \tag{3}$$

where μ is the global mean rating, b_u and b_i are a learned bias for each user and each item, and p_u and q_i are the user's and item's latent factor vectors [4]. Unlike UBCF and IBCF, all of these parameters are learned by stochastic gradient descent on the observed ratings, with the library's default learning rate (0.005) and regularization (0.02). Default settings were retained and no tuning was performed (see Section V-B).

G. Baseline Models

Three non-personalized baseline predictors were implemented to establish a lower bound on acceptable performance: Global Mean, which always predicts the overall average rating across the training set; User Mean, which predicts each user's own average rating regardless of item; and Item Mean, which predicts each item's own average rating regardless of user. A properly functioning personalized CF model is expected to outperform all three; failure to do so would indicate a fundamental implementation defect rather than a genuine modeling limitation.

H. Evaluation Metrics

Two categories of metric were used. Accuracy metrics (Root Mean Square Error, RMSE, and Mean Absolute Error, MAE) measure how close predicted ratings are to actual ratings on the held-out test set. Ranking metrics (Precision@K, Recall@K, and their harmonic mean F1@K) were additionally implemented because RMSE alone does not capture what a recommender system is actually used for: presenting a short, ranked list of items [6]. An item was defined as "relevant" to a user if their true rating for it was 4.0 or higher, the conversion commonly applied to MovieLens ratings [5]. Precision@K measures the fraction of a user's top-K recommendations that are relevant; Recall@K measures the fraction of all relevant items that appear within the top K.

I. Hyperparameter Tuning

Both UBCF and IBCF expose two key hyperparameters: k, the neighborhood size, and `min_support`, the minimum co-rating threshold. A grid search was performed over $k \in \{5, 10, 20, 30, 40\}$ and `min_support` $\in \{1, 3, 5, 10\}$, giving 20 configurations per model and 40 in total, with each configuration evaluated on RMSE and Precision@10 on the test set (no separate validation set was held out; see Section V-B). Because rating-accuracy and ranking-quality do not necessarily optimize together, both the RMSE-optimal and Precision@10-optimal configurations were recorded for each model; Precision@10 was adopted as the primary selection criterion for the final configuration used in all subsequent experiments, since ranking quality is arguably the more practically relevant objective for a recommender system. The RMSE-optimal alternative is reported alongside it in Section IV-C for completeness.

IV. Results

A. Model Comparison

Table I reports RMSE, MAE, Precision@10, and Recall@10 for all six models (the three baselines, tuned UBCF, tuned IBCF, and SVD), evaluated on the same held-out test set of 13,406 ratings.

TABLE I
FINAL COMPARISON OF ALL SIX MODELS ON THE HELD-OUT TEST SET (13,406 RATINGS). "MS" = MIN_SUPPORT.

Model	RMSE	MAE	P@10	R@10
SVD (n_factors=50)	0.8379	0.6435	0.0601	0.0495
UBCF (k=20, ms=10)	0.8430	0.6407	0.0499	0.0453
IBCF (k=30, ms=1)	0.8769	0.6731	0.0562	0.0382
User Mean	0.9128	0.7064	0.0250	0.0246
Item Mean	0.9253	0.7152	0.0365	0.0346
Global Mean	0.9990	0.8000	0.0250	0.0246

Fig. 1 plots the same RMSE and Precision@10 values. All three personalized models (SVD, UBCF, IBCF) outperform all three non-personalized baselines on every metric. On RMSE the gap is large (0.84–0.88 against 0.91–1.00). The ranking-metric differences are small in absolute terms, with

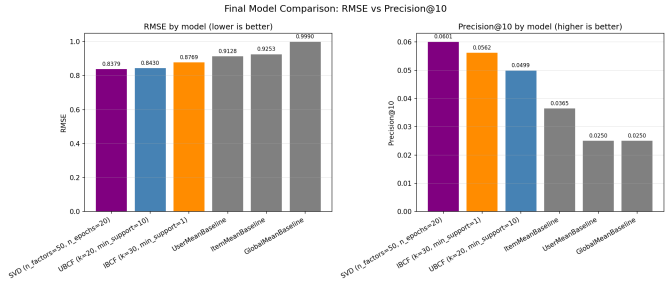


Fig. 1. RMSE and Precision@10 across all six models.

every Precision@10 value at or below 0.06, and were not significance-tested, so they should be read as directional.

SVD achieves the best RMSE, MAE, Precision@10, and Recall@10 simultaneously; this is directionally consistent with Koren et al. [4], who report that matrix factorization tends to outperform neighborhood-based methods on the much larger Netflix Prize dataset, although this project does not independently establish that the same advantage holds at ml-latest-small's scale. The comparison is not an equal one, however: UBCF and IBCF were each tuned in the grid search of Section III-I (20 configurations per model), while SVD was run with default hyperparameters (50 factors, 20 epochs) and no tuning. SVD's advantage in this table should therefore be read as a lower bound on what a tuned SVD model could achieve, not as a fair three-way comparison at matched effort; Section V-B returns to this point.

Between the two neighborhood-based methods, which were tuned comparably to each other, UBCF achieves the lower RMSE while IBCF achieves the higher Precision@10 (0.0562 vs. 0.0499), suggesting that rating accuracy and ranking quality are related but not the same objective, and can favor different models.

B. Statistical Significance of the UBCF–IBCF Difference

To determine whether UBCF's RMSE advantage over IBCF (0.8430 vs. 0.8769) is larger than the sampling variation of the test set, a paired bootstrap test [12] was performed on the two fitted models. The resampling unit was the individual (user, movie) test-set prediction: on each of 1,000 iterations, the same set of prediction indices was drawn with replacement and applied to both models' per-prediction squared-error arrays (a paired bootstrap, so both models are always compared on identical resampled rows), and RMSE was recomputed for each model on that resample. UBCF achieved lower RMSE than IBCF in 100% of the 1,000 resamples, with a mean RMSE difference (IBCF − UBCF) of 0.0335 and a 95% confidence interval of [0.0252, 0.0418], which excludes zero. This supports the conclusion that UBCF's advantage over IBCF is unlikely to be explained by which test ratings happened to be held out.

The test has two caveats. First, the models were not retrained during resampling, so it says nothing about variation from training on a different split; that would require repeated splits or cross-validation (Section V-B). Second, predictions were resampled individually rather than by user, which implicitly treats each prediction as an independent draw, whereas predictions from the same user (or for the same movie) share that user's rating history and neighbor set. Because of this simplification, the reported confidence interval could understate the true sampling variability to some degree. Given the size of the effect (a 100% win rate, and a lower CI bound of 0.0252, about three times the interval's half-width of 0.0083 away from zero), the qualitative conclusion is unlikely to be overturned by a user-level (cluster) bootstrap, but that stronger design is left as future work rather than assumed.

C. Hyperparameter Tuning Results

Table II summarizes the outcome of the 40-configuration grid search described in Section III-I, and Fig. 2 plots RMSE and Precision@10 across the full range of k for each `min_support` value. For both models, the RMSE-optimal and Precision@10-optimal configurations differ, which shows that the two objectives do not have the same optimum, in line with the earlier finding that lower RMSE does not necessarily give better top-N lists [6].

TABLE II
RMSE-OPTIMAL VS. PRECISION@10-OPTIMAL HYPERPARAMETER CONFIGURATIONS. THE PRECISION@10-OPTIMAL CONFIGURATION WAS ADOPTED FOR ALL SUBSEQUENT EXPERIMENTS.

Model / Criterion	k	ms	RMSE	P@10
UBCF — best RMSE	40	5	0.8371	0.0406
UBCF — best P@10 (used)	20	10	0.8430	0.0499
IBCF — best RMSE	20	3	0.8729	0.0522
IBCF — best P@10 (used)	30	1	0.8769	0.0562

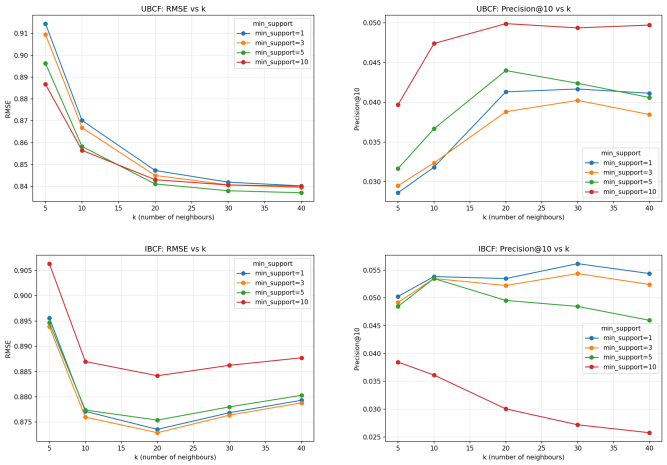


Fig. 2. RMSE and Precision@10 as a function of neighborhood size k, for each min_support value tested. Top row: UBCF. Bottom row: IBCF.

D. Cold-Start and Sparsity Analysis

To characterize behavior under limited data availability (the "cold-start" problem [13], studied here for users with few ratings rather than for new items), each user's training ratings

were randomly truncated to a maximum of 1, 3, 5, 10, and 20 ratings, the models were refitted, and RMSE was measured on the test set (Table III). Test pairs whose movie no longer appears in the truncated training data cannot be scored, so each level is evaluated on a different subset, from 5,974 of the 13,406 test ratings at one rating per user up to all of them at full data. Part of the change across levels therefore reflects which test ratings are scored, not only model quality.

TABLE III
RMSE UNDER SIMULATED COLD-START CONDITIONS

Max ratings/user	UBCF RMSE	IBCF RMSE	n scored
1	1.2605	1.2605	5,974
3	1.0495	1.0571	9,751
5	1.0031	1.0388	11,385
10	0.9531	1.0705	12,732
20	0.9337	1.0924	13,266
Full	0.8430	0.8769	13,406

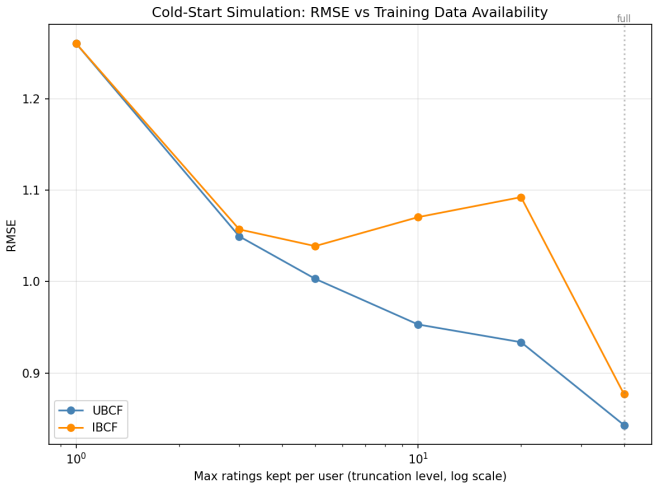


Fig. 3. RMSE vs. training data availability (simulated cold-start).

Fig. 3 plots this relationship. UBCF's RMSE decreases monotonically as more training data becomes available, matching the expected "learning curve" pattern. The identical values at one rating per user are not a coincidence. With a single rating, a user's mean equals that rating, so every user-centered value is zero and all of IBCF's adjusted-cosine similarities vanish; UBCF's Pearson similarities are undefined because no pair reaches ten co-rated items. Both models therefore fall back to predicting the user's one observed rating.

IBCF, however, displays non-monotonic behavior: RMSE improves through 5 ratings/user but then worsens at 10 and 20 before recovering sharply at full data. The leading hypothesis is IBCF's permissive `min_support` = 1 (Section IV-C): as truncation is relaxed, more item pairs obtain a non-zero similarity from only one or two co-raters, which is a thin basis for a similarity estimate, and this noise only averages out once the full data restores co-rating density. Rerunning the sweep with IBCF's RMSE-optimal configuration (k = 20, `min_support` = 3, Table II) reduced the rise after the minimum from 0.0536 to 0.0228, less than half, though a residual increase remained at the 10→20 transition. Because k changed together with `min_support`, this supports the hypothesis without isolating `min_support` as the only cause.

A complementary analysis bucketed individual test predictions by the number of neighbors actually available at prediction time (Table IV). For UBCF, RMSE generally decreases as neighbor support increases, from 1.1087 in the 0–2 bucket (which includes fallback predictions) to 0.8236 in the 11–20 bucket, with one small reversal between the 3–5 and 6–10 buckets. This is broadly consistent with the intuition that predictions backed by more corroborating evidence are more reliable.

TABLE IV
UBCF PREDICTION RMSE BUCKETED BY NEIGHBOR-SUPPORT COUNT[A]

Neighbor count bucket	UBCF RMSE	UBCF n
0–2	1.1087	122
3–5	0.9658	277
6–10	0.9833	1,026
11–20	0.8236	11,981

[a]IBCF populated only two buckets: 11–20 (n = 330, RMSE 1.0021) and >20 (n = 13,076, RMSE 0.8735), so its low-support behavior could not be measured.

E. Scalability Analysis

Model fit time was measured across four training-set sizes (25%, 50%, 75%, 100% of the training data); Fig. 4 plots the result. Both models' fit time remained under 0.15 seconds even at full data size, with negligible growth across the range. This is because the number of distinct users and items is already saturated at 25% of the data, a direct consequence of the ≥20-ratings-per-entity filtering described in Section III-B, so the similarity matrices being computed do not grow further as more ratings are added.

The large wall-clock difference observed during hyperparameter tuning (roughly 5–6 minutes per UBCF configuration vs. 40–70 seconds per IBCF configuration, each covering fitting plus ranking evaluation) was therefore not caused by fitting. Timing `predict()` directly (20 repeated calls on one user–movie pair) gave 0.593 ms per call for UBCF and 0.064 ms for IBCF. This 9.2× difference comes from UBCF's prediction path relying on per-call pandas operations, compared to IBCF's NumPy array lookups. Ranking evaluation calls `predict()` for roughly 900–1,000 candidate movies per user, so across 566 users these latencies imply about 5 minutes for UBCF and about 33 seconds for IBCF, close to the observed tuning times.

Of the two neighborhood-based methods, IBCF therefore scales more favorably to a growing user base in this implementation; for a growing catalog the picture would be expected to reverse at fitting time, since IBCF's item-item similarity matrix grows quadratically with the number of movies. The prediction-latency gap is specific to how the two models were coded here, not a claim about neighborhood-based

collaborative filtering in general; a differently implemented UBCF predictor (for example, one using vectorized NumPy operations throughout, as IBCF already does) could plausibly avoid it.

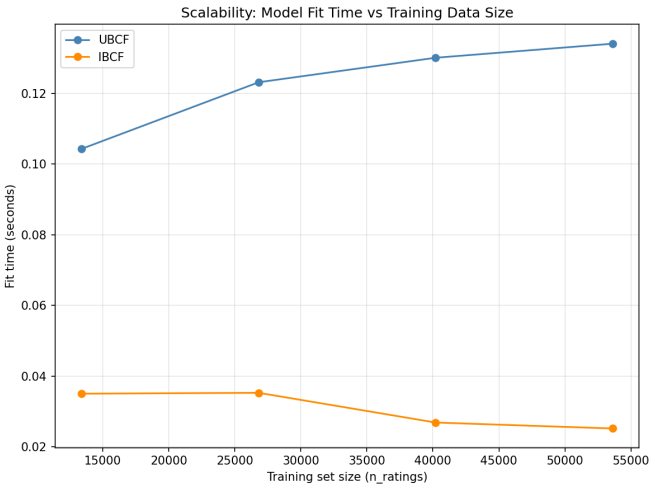


Fig. 4. Model fit time as a function of training set size.

V. Discussion

A. Implementation Challenges

Two implementation defects were found and fixed during development. Both are documented here because they are the kind of mistake that is easy to make when implementing CF algorithms by hand.

First, an early version of IBCF aggregated raw user ratings weighted by signed adjusted-cosine similarities ($\sum \text{sim} \cdot r / \sum |\text{sim}|$) rather than mean-centered deviations. Because adjusted cosine similarity can be negative, this formula allowed an item with strongly negative similarity but a high raw rating to contribute a large negative term to the numerator while still increasing the denominator, systematically pulling predictions toward the bottom of the rating scale; a diagnostic run on the first 20 test ratings showed 9 of them (45%) clipped to the scale minimum (0.5) under this defect, with true ratings for those same items mostly in the 3–5 range. Replacing the formula with item-mean-centered deviations (Eq. (2), Section III-E), consistent with the standard formulation in Sarwar et al. [2], resolved the issue, reducing RMSE from an initial 2.50 to 0.87–0.91 across the tuning grid (0.8769 for the configuration in Table I).

Second, approximately 1.07% of UBCF's raw (pre-clipping) predictions exceeded the upper bound of the rating scale in a pre-tuning configuration (k = 20, `min_support` = 5). A single user examined in detail, who has a high personal mean rating, saturated at 4.1%, about four times the overall rate; with one user this is only an indication, not evidence, that the effect concentrates among high-mean users. This was traced not to a coding error but to a mathematical property of the weighted-deviation formula (Eq. (1)): because it is an additive combination of a mean and unbounded weighted deviations rather than a convex combination of ratings, it is not guaranteed to remain within [0.5, 5.0] when a user's neighbors show strong positive consensus. Clipping to the valid range, as implemented, is a common remedy. A secondary consequence was investigated: when multiple candidate recommendations clip to the same boundary value, their relative order in a top-K recommendation list becomes arbitrary unless explicitly broken. A deterministic tie-breaking rule was added to ensure reproducible, non-arbitrary rankings among such ties: predictions are ordered first by predicted score, then by the number of neighbors supporting that prediction, then by movie ID.

B. Limitations

- Both neighborhood models were tuned via grid search; SVD was evaluated with default hyperparameters (50 factors, 20 epochs) without a comparable tuning phase. Hyperparameter selection can change the outcome of such comparisons substantially [9]; SVD's already-superior performance may understate its true advantage relative to a tuned configuration, a question left open for future work.
- Hyperparameters were selected on the test set: the grid search scored every configuration on the same 13,406 held-out ratings used for the final comparison, and no separate validation set was held out. The reported results of the tuned models are therefore optimistically biased [14], most of all Precision@10, where the selected value is the best of 20 configurations per model. SVD was not tuned, so its results do not carry this bias.
- The dataset's aggressive filtering ($\geq$20 ratings per user and per movie, Section III-B) deliberately excludes the most extreme cold-start cases from the primary evaluation; the cold-start analysis in Section IV-D uses artificial truncation of otherwise-eligible users' histories as a proxy, which may not perfectly represent the behavior of an actual new user or item in a live system.
- The non-monotonic behavior of IBCF in the cold-start analysis (Section IV-D) was not fully resolved: a smaller effect persisted after raising `min_support` to 3, and because k also changed in that comparison, the two factors were not isolated.
- The ranking metrics carry two caveats. Relevance was fixed at a rating of at least 4.0 and no sensitivity analysis was run, so results may differ under other thresholds. The candidate sets also differed across models: UBCF's recommendation function excludes movies with fewer than 20 ratings in the training data, while IBCF, SVD, and the baselines rank all unrated movies, and the effect of this on the Precision@10 comparison was not measured.
- The statistical evidence is limited. Only the RMSE difference between UBCF and IBCF was significance-tested; the Precision@10 and Recall@10 differences, including IBCF's advantage over UBCF on Precision@10, were not. All results come from a single fixed train/test split, with the bootstrap applied post hoc to the fitted models; k-

fold cross-validation was not performed and would show the variance across data partitions more completely.

VI. Conclusion and Future Work

This project implemented and evaluated two collaborative filtering approaches, User-Based and Item-Based CF, without a recommender library, alongside three non-personalized baselines and an SVD comparison. Both personalized neighborhood models outperformed the non-personalized baselines on every metric, clearly so on RMSE, and the bootstrap test showed that UBCF's RMSE advantage over IBCF, although numerically small ($\approx$0.034), is larger than test-set sampling variation would explain. SVD had the best overall numbers, in line with prior work on matrix factorization, although it was not tuned. The cold-start and neighbor-support experiments confirmed that neighborhood methods are sensitive to data sparsity: UBCF degraded predictably as data was removed, while IBCF's behavior was more complex and only partially explained by its permissive `min_support` setting. The scalability analysis traced the practical speed gap between UBCF and IBCF to prediction-time implementation choices (pandas vs. NumPy); since no alternative UBCF implementation was tested, this does not show that the gap is inherent to neighborhood-based methods.

Future work could extend this study in several directions: selecting hyperparameters on a separate validation set, running a comparable hyperparameter search for the SVD model, using k-fold cross-validation to assess variance across data partitions, running a sensitivity analysis on the relevance threshold used for the ranking metrics, and further isolating the unexplained part of IBCF's cold-start behavior noted in Section V-B. The implementation, including 133 automated unit and integration tests, a reproducible pipeline, and an interactive demonstration interface, is available in the project's version-controlled repository.

References

[1] P. Resnick, N. Iacovou, M. Suchak, P. Bergstrom, and J. Riedl, "GroupLens: An Open Architecture for Collaborative Filtering of Netnews," in *Proc. ACM Conf. Computer Supported Cooperative Work (CSCW)*, 1994, pp. 175–186.

[2] B. Sarwar, G. Karypis, J. Konstan, and J. Riedl, "Item-Based Collaborative Filtering Recommendation Algorithms," in *Proc. 10th Int. Conf. World Wide Web (WWW)*, 2001, pp. 285–295.

[3] J. L. Herlocker, J. A. Konstan, A. Borchers, and J. Riedl, "An Algorithmic Framework for Performing Collaborative Filtering," in *Proc. 22nd Annu. Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, 1999, pp. 230–237.

[4] Y. Koren, R. Bell, and C. Volinsky, "Matrix Factorization Techniques for Recommender Systems," *Computer*, vol. 42, no. 8, pp. 30–37, Aug. 2009.

[5] J. L. Herlocker, J. A. Konstan, L. G. Terveen, and J. T. Riedl, "Evaluating Collaborative Filtering Recommender Systems," *ACM Trans. Information Systems*, vol. 22, no. 1, pp. 5–53, Jan. 2004.

[6] P. Cremonesi, Y. Koren, and R. Turrin, "Performance of Recommender Algorithms on Top-N Recommendation Tasks," in *Proc. 4th ACM Conf. Recommender Systems (RecSys)*, 2010, pp. 39–46.

[7] A. N. Nikolakopoulos, X. Ning, C. Desrosiers, and G. Karypis, "Trust Your Neighbors: A Comprehensive Survey of Neighborhood-Based Methods for Recommender Systems," in *Recommender Systems Handbook*, 3rd ed., F. Ricci, L. Rokach, and B. Shapira, Eds. New York, NY, USA: Springer, 2022, pp. 39–89.

[8] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, "Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches," in *Proc. 13th ACM Conf. Recommender Systems (RecSys)*, 2019, pp. 101–109.

[9] S. Rendle, W. Krichene, L. Zhang, and J. Anderson, "Neural Collaborative Filtering vs. Matrix Factorization Revisited," in *Proc. 14th ACM Conf. Recommender Systems (RecSys)*, 2020, pp. 240–248.

[10] F. M. Harper and J. A. Konstan, "The MovieLens Datasets: History and Context," *ACM Trans. Interactive Intelligent Systems*, vol. 5, no. 4, article 19, Dec. 2015.

[11] N. Hug, "Surprise: A Python Library for Recommender Systems," *Journal of Open Source Software*, vol. 5, no. 52, p. 2174, 2020.

[12] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman & Hall, 1993.

[13] A. I. Schein, A. Popescul, L. H. Ungar, and D. M. Pennock, "Methods and Metrics for Cold-Start Recommendations," in *Proc. 25th Annu. Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, 2002, pp. 253–260.

[14] G. C. Cawley and N. L. C. Talbot, "On Over-fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation," *Journal of Machine Learning Research*, vol. 11, pp. 2079–2107, 2010.